Working with Browser Events for Interactive Visualisations

Visualisation and Sensing BSC2 2025 | Tom Armitage | t.armitage@arts.ac.uk

In this exercise, you'll add code to a map that uses Leaflet to draw a map of meteorite discoveries and sightings over history.

You'll refresh your knowledge of events by:

- attaching events to button click
- working with the Navigation API
- filtering data by using browser events on different input elements

Then, you'll go on to implement them with a Fetch-based API.

You'll start by refactoring the code to put the Javascript in an external file. Then, you'll start tracking browser events to filter that data, as well as working with the Navigation API, before going on to look at requesting data from a JSON API.

Always prefer typing out code, rather than copy-pasting - unless the text tells you otherwise.

Timing

This workshop is designed to take an hour, including lecturer walkthrough.

Requirements

VS Code with Live Server extension.

Getting Started

Open this directory in Visual Studio Code, and run VS Code Live Server. You must access

your code through a server running at localhost - you can't just double-click `index.html` in Explorer or Finder.

You should work in `index.html`.

Start Live Server running in this directory.

At `http://localhost:5500` you should see a page with the title " Interactive Visualisation with Events"; a map should load with a selection of blue dots on it.

> **Notes on the template HTML**
>
> The `index.html` template contains a few things already:
>
> - it loads the Leaflet JS library
> - it loads the CSS needed for Leaflet's styles to work
> - it includes some simple CSS, primarily to set the element with id `map` to a fixed height. (Leaflet requires map divs to have set heights.)

1. Review the starting code.

Task: Review the code in `index.html`. *Only spend a few minutes on this - it's just to familiarise yourself with what's in the page.*

In particular:

- review the main `<body>` of HTML: what elements there are, particularly in `<div class='controls'>`.
- review the Javascript in the script tags at the end of the page. In particular:
  - start where at the comment `begin main display code`. This runs first.
  - note which variables have been set up at a global level - these will be available to all functions defined
  - finally, review the `drawMeteorites` function.

2. Handling button clicks

Task: Implement the reset button; it should reset the map back to its original zoom and position.

Bind a function to the act of clicking the reset button. The reset button has the `id` of `reset`. You can use `document.getElementById` to find the reset button, and then `addEventListener` to bind the click event to an anonymous function.

This function should set the map back to its center (`[0,0]`) and its original zoom level (2). `map` is assigned to a global variable, and the `setView` method on it will let you do

exactly that.

3. Advanced Button Clicks: using the Browser Geolocation API.

Task: Implement the Find Me! button; it should set the map to the user's current location at a suitable zoom level.

If you've ever used a website on your computer or phone that asks permission to know your current location, and then uses it: that's using the **Geolocation API** of modern browsers.

The MDN Guide to the Geolocation API should get you up and running quickly. Add another event listener, to the "Find Me" button, that gets the user's current position, and then sets the map that location at a higher zoom level (start with **13**, but choose a level you think works for yourself).

4. Listening to input change

Task: filter the data to only show meterorite discoveries between the years in the two input boxes.

Write two more listeners, one for each input element. This time, listen to the `change` element; you'll be able to get the current value of the element that fired the callback via the `value` property of the event's `target`:

```
document.getElementByID("example").addEventListener("change", (event) => {
  console.log(`The value is now ${event.target.value}`);
});
```

When, *exactly*, does the change event fire?

Use this event to adjust the range of meteorites on display. The original developer of this file has already created a useful `drawMeteorites` function you can call to do this. **You will need to edit this function at the two places where it says TODO to do the filtering.**

5. Replacing inputs text fields with sliders.

Task: Make the input fields for start/end year sliders.

There are other input types beyond ranges in HTML. To make the text inputs sliders:

- add three *attributes* to each of the year input elements:
    - set `min` to the lowest value the slider should represent
    - set `max` to the highest value the slider should represent

    - set `type` to `range`

Try your sliders now. When does the `change` event fire?

It'd be better if it fired continuously as the user dragged, rather than at the end. Change your listener to listen for `input` events, rather than `change`.

The range slider no longer shows the numerical value of the year.

Task: Set the `innerText` of the currently unused `<span>` element next to the input to display the appropriate numeric value.

6. Stretch goal: working with an API

This is a stretch goal for when you've finished all the above.

Currently, the code requests the data from the JSON data file, and filters it in memory. Another approach is to request it from an API, and use the API to filter it.

The API is, for the duration of this class, accessible via a `GET` request to https://api-rough-butterfly-8322.fly.dev/meteorites

By default, you'll get *all* data back. The `startyear` and `endyear` query parameters - both optional - will filter data for you. Eg: https://api-rough-butterfly-8322.fly.dev/meteorites?startyear=1900&endyear=1910

To use this API in your code:

- update the `fetch` request to query the API endpoint.
- fire a `fetch` when the slider finishes moving (`change`) rather than on `input` (you don't want to get rate limited!).
- the filtering of years should now happen when fetching data, not in the draw function.

7. Stretch goal: extracting JavaScript to a separate own file.

Until now, we've been putting our Javascript in script tags at the end of the HTML file. Our HTML file is getting rather large. Extract the javascript to a new file (eg: viz.js) and reference it in the `<head>` of the page, just like our other libraries.

If you do this with no changes, it probably won't work. Why not?

You can solve the issues that emerge by listening for the `DOMContentLoaded` event on `document`. I'll let you work out the rest of that yourself.